UTN — INSPT
Tecnicatura Universitaria en Informática Aplicada

ESPECIFICACIONES TÉCNICAS PREVIAS

Proyecto de Base de Datos — MySQL 8.x

Materia: Estructura y Base de Datos

Nombre del proyecto:	Vivero
Versión:	1.0
Fecha:	24/10/2026
Autor / Equipo:	Grupo 3: • Álvarez Varone, Guido Martín • Deluca, Nicolás Andrés • Morea, Laura Elena
Comisión:	2.603
Docente:	Ing. Alejandro Jorge Behringer/ Lic. Jonathan Gabriel Pécora
Auxiliar docente:	Prof. Juan Carlos Capia
Motor de Base de Datos:	MySQL 8.x
Tipo de Base de Datos:	Relacional

INDICE

1. Identificador del proyecto 3
2. Descripción de la necesidad del usuario 4
    2.1. Contexto general 4
    2.2. Objetivo del sistema 4
    2.3. Alcance del sistema 4
3. Relevamiento de requerimientos 5
    3.1. Requerimientos funcionales 5
    3.2. Requerimientos no funcionales 5
4. Identificación de entidades y atributos 6
    4.1. Listado de entidades 6
    4.2. Atributos 7
5. Definición de claves 14
    5.1. Claves primarias 14
    5.2. Claves foráneas 15
6. Modelo conceptual 16
    6.1. Diagrama de Entidad-Relación 16
    6.2. Esquema Entidad-Relación por tablas 16
7. Modelo lógico 17
8. Normalización 19
    8.1 Resumen del análisis de la normalización 19
9. Modelo físico – MySQL 20
    9.1. Tipos de datos y restricciones 20
    9.2. Script SQL de creación 20
10. Validación del modelo 31
    10.1. Casos de prueba 31
    10.2. Consultas SQL de validación 34
11. Conclusión técnica 35
    11.1. Mejoras futuras 35

1. Identificación del Proyecto

Nombre del proyecto:	Vivero
Versión:	1.0
Fecha:	24/10/2026
Autor / Equipo:	Grupo 3: • Álvarez Varone, Guido Martín • Deluca, Nicolás Andrés • Morea, Laura Elena
Motor de Base de Datos:	MySQL 8.x
Tipo de Base de Datos:	Relacional

2. Descripción de la Necesidad del Usuario

2.1 Contexto general

El proyecto se enmarca en la digitalización y optimización de la gestión de un vivero comercial dedicado a la comercialización minorista de especies vegetales (plantas de interior y exterior, arbustos, árboles frutales y ornamentales), así como a la venta de insumos complementarios (sustratos, fertilizantes, macetas y herramientas de jardinería).
Actores / Usuarios del Sistema:

- *Vendedor:* Encargado de la gestión de clientes, transacciones, compra de mercadería, control general de finanzas y altas/bajas de productos.
- *Cliente:* Persona a la que se le realiza la venta.
- *Proveedor:* Encargado del ingreso de nueva mercadería.

Situación Actual: Actualmente, la organización opera bajo una metodología rudimentaria basada en registros en papel y planillas de cálculo descentralizadas. Esta situación genera graves problemáticas operativas: inconsistencia en la actualización de precios, desfasajes frecuentes en el control de stock, falta de trazabilidad en las ventas y demoras significativas en la generación de informes comerciales para la toma de decisiones.

2.2 Objetivo del sistema

El objetivo principal del proyecto es diseñar e implementar una base de datos relacional robusta y normalizada que centralice, ordene y asegure la integridad de toda la información crítica del vivero. El sistema busca resolver el problema de la fragmentación de datos y la falta de control en tiempo real sobre el inventario y las transacciones comerciales.
A nivel técnico y operativo, la base de datos eliminará la redundancia de datos mediante la correcta aplicación de formas normales. Asimismo, permitirá automatizar el seguimiento del stock disponible, registrar de manera estructurada las ventas y los pagos, y facilitar consultas complejas y reportes gerenciales (tales como productos con bajo stock, historial de compras por cliente y niveles de ventas por período) para optimizar la gestión administrativa y comercial del establecimiento.

2.3 Alcance del sistema

Incluye	No incluye
Gestión de inventario	Sistema de gestión del pago online
Gestión de proveedores y clientes	Sistema de automatización de riegos
Registro de compras y ventas	Servicios a domicilio de manutención del jardín

3. Relevamiento de Requerimientos

3.1 Requerimientos funcionales

N°	Requerimiento funcional
RF-01	Registrar altas, bajas y modificaciones de productos (plantas, sustratos, fertilizantes e insumos) en el inventario del vivero.
RF-02	Gestionar el registro, actualización y consulta de la información de los clientes y proveedores de la organización.
RF-03	Registrar transacciones de venta, asociando el detalle de los productos adquiridos, las cantidades, el cliente y el método de pago correspondiente.
RF-04	Consultar en tiempo real la disponibilidad de stock de cada artículo para agilizar la atención comercial.
RF-05	Generar reportes gerenciales de ventas por período, historiales de compra por cliente y alertas automáticas de stock crítico.

3.2 Requerimientos no funcionales

N°	Requerimiento no funcional
RNF-01	Las consultas de búsqueda de productos por nombre o categoría, así como el procesamiento de la venta se agilizan en gran medida garantizando fluidez en la atención al cliente.
RNF-02	Garantizar la consistencia transaccional y la prevención de anomalías en la integridad de los datos y la aplicación de restricciones de integridad referencial, en cuanto a claves primarias y foráneas.
RNF-03	Dimensionar la base de datos, de forma tal que permita gestionar un volumen inicial proyectado de 50.000 registros anuales de ventas y un catálogo activo de hasta 10.000 productos y lotes de insumos, permitiendo la escalabilidad del sistema.

4. Identificación de Entidades y Atributos

4.1 Listado de entidades

N°	Entidad	Tipo (Fuerte/Débil)	Descripción breve
1	vendedor	fuerte	Almacena los datos personales, de contacto y de identificación del personal encargado de concretar las ventas en el vivero.
2	cliente	fuerte	Registra la información personal y de contacto de los clientes que efectúan compras en el establecimiento.
3	proveedor	fuerte	Contiene la información fiscal (CUIT), razón social y canales de contacto de los proveedores de mercadería e insumos.
4	producto	fuerte	Entidad central (supertipo) que consolida el control de existencias, el stock mínimo, los precios de compra y de venta y el estado (activo o dado de baja) de cada artículo comercial.
5	planta	fuerte (subtipo de producto)	Subtipo de producto que registra el nombre, la categoría, las características botánicas y los requerimientos de cuidado de cada planta.
6	insumo	fuerte (subtipo de producto)	Subtipo de producto que registra el catálogo de herramientas, sustratos y productos no vegetales.
7	venta	fuerte	Registra la cabecera de las transacciones comerciales realizadas (fecha, forma de pago, total), vinculando obligatoriamente al cliente y al vendedor que la efectúan.
8	detalle_venta	débil	Entidad intermedia entre venta y producto, especificando la cantidad, el precio unitario y el subtotal de cada ítem vendido.

N°	Entidad	Tipo (Fuerte/Débil)	Descripción breve
9	compra	fuerte	Registra los encabezados de los pedidos de reabastecimiento o adquisiciones realizadas a los proveedores (fecha, forma de pago, total facturado).
10	detalle_compra	débil	Entidad intermedia entre compra y producto, especificando la cantidad, el precio unitario y el subtotal de cada ítem comprado.

4.2 Atributos

Tabla vendedor

Atributo	Clave	Tipo de dato	Tamaño	Nulabilidad	Dominio
id_vendedor	**PK**	INT	4 bytes	NOT NULL	Entero positivo autoincremental. Identifica de forma única al vendedor.
nombre		VARCHAR	50 caracteres	NOT NULL	Texto con el nombre de pila del vendedor.
apellido		VARCHAR	50 caracteres	NOT NULL	Texto con el apellido del vendedor.
dni	**UQ**	VARCHAR	12 caracteres	NOT NULL	Solo dígitos, sin puntos ni espacios. No se repite entre vendedores.
telefono		VARCHAR	15 caracteres	NOT NULL	Teléfono de contacto. Dígitos, con “+” inicial opcional, entre 8 y 14 dígitos.
mail		VARCHAR	100 caracteres	NULL	Dirección de correo electrónico (con “@” y dominio). No se permiten espacios ni doble “@”.
direccion		VARCHAR	100 caracteres	NULL	Texto con el domicilio (calle, número y localidad).

Tabla cliente

Atributo	Clave	Tipo de dato	Tamaño	Nulabilidad	Dominio
id_cliente	**PK**	INT	4 bytes	NOT NULL	Entero positivo autoincremental. Identifica de forma única al cliente.
nombre		VARCHAR	50 caracteres	NULL	Texto con el nombre de pila del cliente.
apellido		VARCHAR	50 caracteres	NULL	Texto con el apellido del cliente.
dni	**UQ**	VARCHAR	12 caracteres	NULL	Dígitos y letras, sin puntos ni espacios (admite pasaportes y documentos extranjeros). Entre 6 y 12 caracteres. No se repite entre clientes.
telefono		VARCHAR	15 caracteres	NULL	Teléfono de contacto. Dígitos, con “+” inicial opcional, entre 8 y 14 dígitos.

Tabla proveedor

Atributo	Clave	Tipo de dato	Tamaño	Nulabilidad	Dominio
id_proveedor	**PK**	INT	4 bytes	NOT NULL	Entero positivo autoincremental. Identifica de forma única al proveedor.
nombre_empresa		VARCHAR	100 caracteres	NOT NULL	Texto con la razón social o nombre comercial.
cuit	**UQ**	VARCHAR	11 caracteres	NOT NULL	Exactamente 11 dígitos, sin guiones. No se repite entre proveedores.
telefono		VARCHAR	15 caracteres	NOT NULL	Teléfono de contacto. Dígitos, con “+” inicial opcional, entre 8 y 14 dígitos.

Tabla producto

Atributo	Clave	Tipo de dato	Tamaño	Nulabilidad	Dominio
id_producto	**PK**	INT	4 bytes	NOT NULL	Entero positivo autoincremental. Identifica de forma única el producto.
cantidad_stock		INT	4 bytes	NOT NULL	Entero mayor o igual a 0. Unidades disponibles (valor por defecto: 0). Lo actualiza la base con cada compra y venta.
stock_minimo		INT	4 bytes	NOT NULL	Entero mayor o igual a 0 (valor por defecto: 0). Si el stock queda por debajo, el producto aparece en la alerta de stock crítico.
precio_compra		DECIMAL	10 dígitos (2 decimales)	NULL	Número decimal mayor o igual a 0. Costo por unidad de la última compra (lo actualiza la base al registrar una compra). NULL mientras el producto no tenga compras (valor por defecto).
precio_venta		DECIMAL	10 dígitos (2 decimales)	NOT NULL	Número decimal mayor o igual a 0. Precio de venta al público por unidad.
activo		BOOLEAN	1 byte	NOT NULL	TRUE = producto a la venta; FALSE = dado de baja (baja lógica, no se puede vender). Valor por defecto: TRUE.

Tabla planta

Atributo	Clave	Tipo de dato	Tamaño	Nulabilidad	Dominio
id_planta	**PK**	INT	4 bytes	NOT NULL	Entero positivo autoincremental. Identifica de forma única la planta.
nombre		VARCHAR	50 caracteres	NOT NULL	Texto con el nombre de la planta.
tipo_planta		ENUM	1 byte	NOT NULL	Valores permitidos: 'suculenta', 'cactus', 'aromática', 'frutal', 'ornamental', 'arbusto', 'árbol', 'flor de estación'.
de_interior		BOOLEAN	1 byte	NOT NULL	TRUE = planta de interior; FALSE = planta de exterior.
descripcion		VARCHAR	100 caracteres	NULL	Texto con la descripción general de la planta.
nivel_de_luz		ENUM	1 byte	NOT NULL	Valores permitidos: 'bajo', 'medio', 'alto'.
riego_cada_dias		SMALLINT	2 bytes	NOT NULL	Entero entre 1 y 365. Cada cuántos días se riega.
id_producto	**FK, UQ**	INT	4 bytes	NOT NULL	Valor existente en producto. No se repite: cada producto tiene a lo sumo una planta y no puede ser también un insumo.

Tabla insumo

Atributo	Clave	Tipo de dato	Tamaño	Nulabilidad	Dominio
id_insumo	**PK**	INT	4 bytes	NOT NULL	Entero positivo autoincremental. Identifica de forma única el insumo.

Atributo	Clave	Tipo de dato	Tamaño	Nulabilidad	Dominio
nombre		VARCHAR	50 caracteres	NOT NULL	Texto con el nombre del insumo.
descripcion		VARCHAR	100 caracteres	NULL	Texto con la descripción general del insumo.
id_producto	**FK, UQ**	INT	4 bytes	NOT NULL	Valor existente en producto. No se repite: cada producto tiene a lo sumo un insumo y no puede ser también una planta.

Tabla venta

Atributo	Clave	Tipo de dato	Tamaño	Nulabilidad	Dominio
id_venta	**PK**	INT	4 bytes	NOT NULL	Entero positivo autoincremental. Identifica de forma única la venta.
fecha		DATE	3 bytes	NOT NULL	Fecha válida, no posterior a la fecha actual (lo controla un trigger). Valor por defecto: la fecha del día.
forma_de_pago		ENUM	1 byte	NOT NULL	Valores permitidos: 'efectivo', 'débito', 'crédito', 'transferencia'.
total_facturado		DECIMAL	10 dígitos (2 decimales)	NOT NULL	Número decimal mayor o igual a 0. Importe total de la venta: suma de los subtotales de sus detalles (lo calcula la base).
id_cliente	**FK**	INT	4 bytes	NOT NULL	Valor existente en cliente. Las ventas a clientes no identificados se asignan al cliente “Consumidor final”.

Atributo	Clave	Tipo de dato	Tamaño	Nulabilidad	Dominio
id_vendedor	**FK**	INT	4 bytes	NOT NULL	Valor existente en vendedor.

Tabla detalle_venta

Atributo	Clave	Tipo de dato	Tamaño	Nulabilidad	Dominio
id_detalle_venta	**PK**	INT	4 bytes	NOT NULL	Entero positivo autoincremental. Identifica de forma única el detalle de la venta.
id_venta	**FK**	INT	4 bytes	NOT NULL	Valor existente en venta.
id_producto	**FK**	INT	4 bytes	NOT NULL	Valor existente en producto. No se repite dentro de una misma venta (UNIQUE id_venta, id_producto).
cantidad		INT	4 bytes	NOT NULL	Entero mayor a 0. Unidades vendidas del producto.
precio_unitario		DECIMAL	10 dígitos (2 decimales)	NOT NULL	Número decimal mayor o igual a 0. Precio cobrado por unidad al momento de la venta.
subtotal		DECIMAL	10 dígitos (2 decimales)	NOT NULL	Número decimal mayor o igual a 0. Resulta de cantidad × precio_unitario (lo calcula la base).

Tabla compra

Atributo	Clave	Tipo de dato	Tamaño	Nulabilidad	Dominio
id_compra	**PK**	INT	4 bytes	NOT NULL	Entero positivo autoincremental. Identifica de forma única la compra.
fecha		DATE	3 bytes	NOT NULL	Fecha válida, no posterior a la fecha actual (lo controla un trigger). Valor por defecto: la fecha del día.

Atributo	Clave	Tipo de dato	Tamaño	Nulabilidad	Dominio
forma_de_pago		ENUM	1 byte	NOT NULL	Valores permitidos: 'efectivo', 'débito', 'crédito', 'transferencia'.
total_facturado		DECIMAL	10 dígitos (2 decimales)	NOT NULL	Número decimal mayor o igual a 0. Importe total de la compra: suma de los subtotales de sus detalles (lo calcula la base).
id_proveedor	**FK**	INT	4 bytes	NOT NULL	Valor existente en proveedor.

Tabla detalle_compra

Atributo	Clave	Tipo de dato	Tamaño	Nulabilidad	Dominio
id_detalle_compra	**PK**	INT	4 bytes	NOT NULL	Entero positivo autoincremental. Identifica de forma única el renglón.
cantidad		INT	4 bytes	NOT NULL	Entero mayor a 0. Unidades compradas del producto.
precio_unitario		DECIMAL	10 dígitos (2 decimales)	NOT NULL	Número decimal mayor o igual a 0. Precio pagado por unidad en esta compra; pasa a ser el precio_compra del producto.
subtotal		DECIMAL	10 dígitos (2 decimales)	NOT NULL	Número decimal mayor o igual a 0. Resulta de cantidad × precio_unitario (lo calcula la base).
id_producto	**FK**	INT	4 bytes	NOT NULL	Valor existente en producto. No se repite dentro de una misma compra (UNIQUE id_compra, id_producto).
id_compra	**FK**	INT	4 bytes	NOT NULL	Valor existente en compra.

5. Definición de Claves

5.1 Claves primarias

Entidad	Clave primaria (PK)	Justificación
vendedor	**id_vendedor**	Clave sustituta. El dni sería una clave natural candidata, pero puede cargarse con errores o corregirse; con un id propio la identidad del vendedor no depende de datos personales. El dni se conserva como UNIQUE.
cliente	**id_cliente**	Clave sustituta. El dni puede corregirse y no siempre es un DNI argentino (pasaportes, documentos extranjeros), y el historial de compras no debería depender de él. El dni se conserva como UNIQUE.
proveedor	**id_proveedor**	Clave sustituta. El cuit es una clave natural candidata, pero puede cargarse con errores o cambiar (corrección o cambio de razón social). El cuit se conserva como UNIQUE.
producto	**id_producto**	Clave sustituta. La tabla no tiene un código de producto natural: el stock y los precios cambian, y el nombre está en planta o insumo, no en producto.
planta	**id_planta**	Clave sustituta. El nombre puede repetirse (variedades, tamaños) y puede corregirse; ningún atributo o combinación es a la vez estable y único.
insumo	**id_insumo**	Clave sustituta. El nombre puede repetirse (mismo producto en distintas presentaciones) y puede modificarse; no existe un atributo natural estable y único.
venta	**id_venta**	Clave sustituta. No tiene un atributo natural que la identifique (fecha, forma de pago y total pueden repetirse). El número autoincremental actúa como número de operación.
detalle_venta	**id_detalle_venta**	Clave sustituta. La alternativa natural sería la clave compuesta (id_venta, id_producto); se conserva como UNIQUE para que un producto aparezca una sola vez por venta (si se llevan más unidades, se aumenta la cantidad).
compra	**id_compra**	Clave sustituta. No tiene un atributo natural que la identifique (fecha, forma de pago y total pueden repetirse). El número autoincremental actúa como número de operación.
detalle_compra	**id_detalle_compra**	Clave sustituta. La alternativa natural sería la clave compuesta (id_compra, id_producto); se conserva como UNIQUE para que un producto aparezca una sola vez por compra.

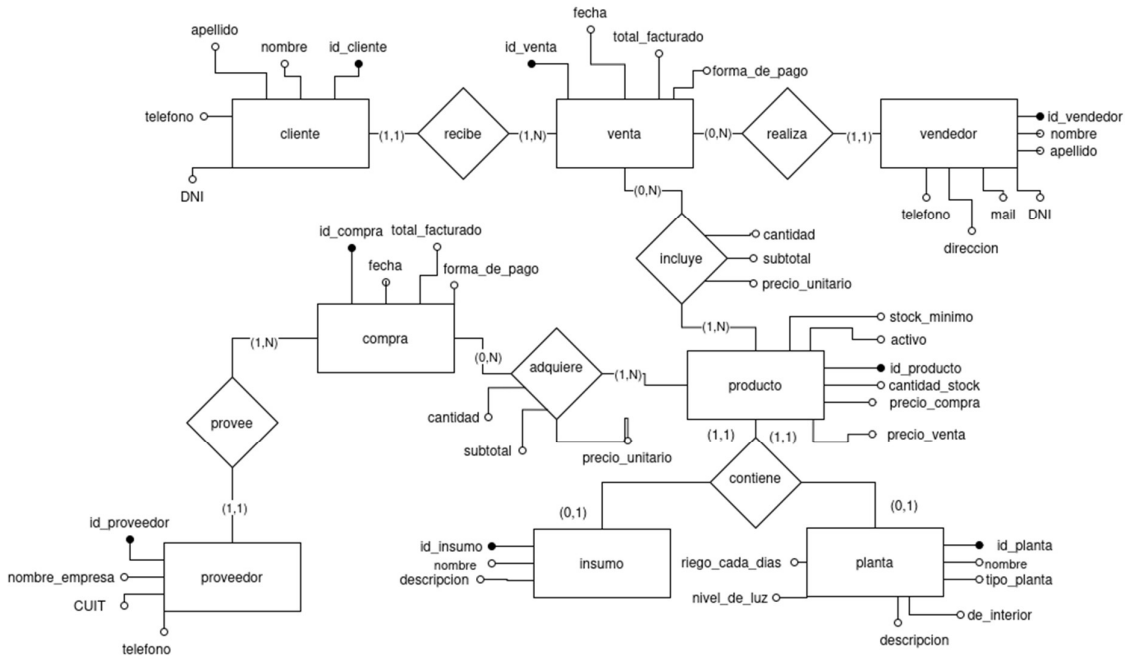
5.2 Claves foráneas y relaciones

Tabla	Clave foránea (FK)	Tabla referenciada	ON DELETE / ON UPDATE
venta	**id_cliente**	cliente (id_cliente)	RESTRICT / CASCADE
venta	**id_vendedor**	vendedor (id_vendedor)	RESTRICT / CASCADE
detalle_venta	**id_venta**	venta (id_venta)	CASCADE / CASCADE
detalle_venta	**id_producto**	producto (id_producto)	RESTRICT / CASCADE
planta	**id_producto**	producto (id_producto)	CASCADE / CASCADE
insumo	**id_producto**	producto (id_producto)	CASCADE / CASCADE
compra	**id_proveedor**	proveedor (id_proveedor)	RESTRICT / CASCADE
detalle_compra	**id_compra**	compra (id_compra)	CASCADE / CASCADE
detalle_compra	**id_producto**	producto (id_producto)	RESTRICT / CASCADE

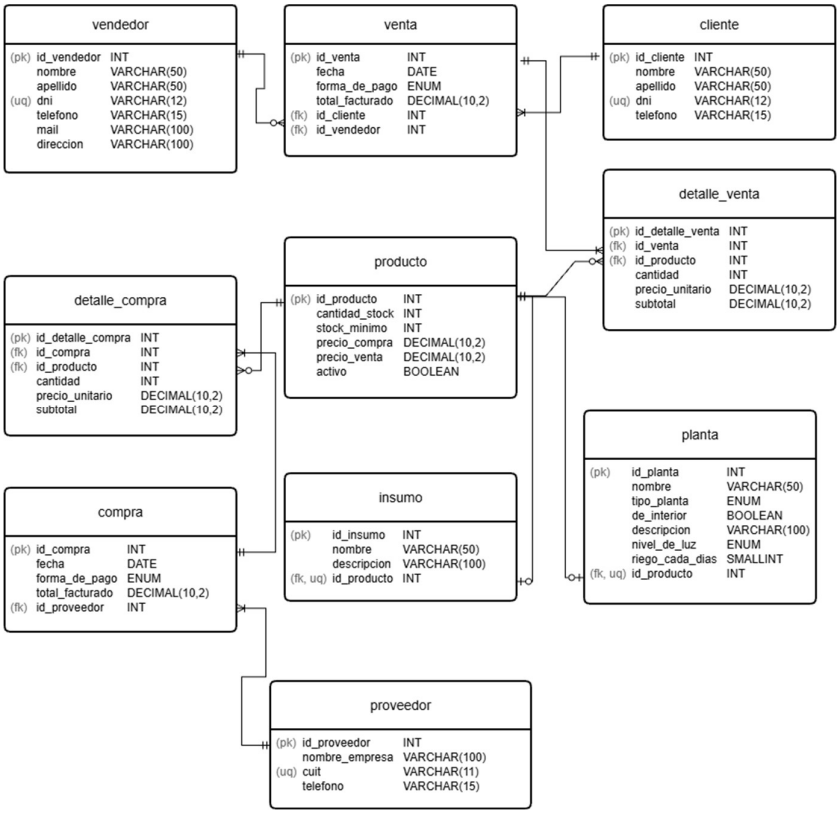
En MySQL las acciones en cascada no disparan triggers. Por eso, al borrar una venta o una compra, los triggers trg_venta_bd y trg_compra_bd ajustan el stock antes de que la cascada elimine sus detalles. Un producto que ya tiene ventas o compras no se puede borrar (RESTRICT): se da de baja con activo = FALSE.

6. Modelo Conceptual

6.1 Diagrama de Entidad-Relación



6.2 Esquema Entidad-Relación por tablas



7. Modelo Lógico

Entidad / Relación del DER	Tabla resultante	Cómo se resolvió (FK / tabla intermedia / herencia)
Entidad: Vendedor	**vendedor**	Tabla propia con PK sustituta id_vendedor. No depende de ninguna otra entidad.
Entidad: Cliente	**cliente**	Tabla propia con PK sustituta id_cliente. No depende de ninguna otra entidad.
Entidad: Proveedor	**proveedor**	Tabla propia con PK sustituta id_proveedor. No depende de ninguna otra entidad.
Entidad: Producto	**producto**	Tabla propia con PK sustituta id_producto. Es el supertipo de la especialización: guarda los datos comunes a plantas e insumos (stock, stock mínimo, precios y estado).
Entidad: Venta	**venta**	Tabla propia con PK sustituta id_venta. Sus relaciones con cliente y vendedor se resuelven con FK dentro de la misma tabla.
Entidad: Compra	**compra**	Tabla propia con PK sustituta id_compra. Su relación con proveedor se resuelve con una FK dentro de la misma tabla.
Entidad: Detalle de venta/ Relación: Incluye	**detalle_venta**	Su existencia depende de una venta. Tabla con PK sustituta id_detalle_venta y FK obligatoria a venta. Es la tabla intermedia de la relación N:M Venta–Producto.
Entidad: Detalle de compra/ Relación: Adquiere	**detalle_compra**	Su existencia depende de una compra. Tabla con PK sustituta id_detalle_compra y FK obligatoria a compra. Es la tabla intermedia de la relación N:M Compra–Producto.
Entidad: Planta	**planta**	Tabla propia con PK sustituta id_planta. Se vincula 1:1 con su producto mediante la FK id_producto, que es UNIQUE.
Entidad: Insumo	**insumo**	Tabla propia con PK sustituta id_insumo. Se vincula 1:1 con su producto mediante la FK id_producto, que es UNIQUE.
Relación 1:N: Vendedor realiza Venta	**venta**	FK. Se agregó id_vendedor en venta, que referencia a vendedor.id_vendedor.
Relación 1:N: Cliente recibe Venta	**venta**	FK. Se agregó id_cliente en venta, que referencia a cliente.id_cliente.
Relación 1:N: Proveedor provee Compra	**compra**	FK. Se agregó id_proveedor en compra, que referencia a proveedor.id_proveedor.
Relación N:M: Venta – Producto (incluye)	**detalle_venta**	Tabla intermedia. Contiene las FK id_venta e id_producto, más los atributos propios de la relación: cantidad, precio_unitario y subtotal.

Entidad / Relación del DER	Tabla resultante	Cómo se resolvió (FK / tabla intermedia / herencia)
Relación N:M: Compra – Producto (adquiere)	**detalle_compra**	Tabla intermedia. Contiene las FK id_compra e id_producto, más los atributos propios de la relación: cantidad, precio_unitario y subtotal.
Relación 1:1: Producto – Planta (contiene)	**planta**	Estrategia de una tabla por subtipo. planta incluye la FK id_producto (UNIQUE) que referencia a producto.id_producto; los atributos propios de las plantas quedan en planta. Un trigger impide que el mismo producto sea también un insumo.
Relación 1:1: Producto – Insumo (contiene)	**insumo**	Estrategia de una tabla por subtipo. insumo incluye la FK id_producto (UNIQUE) que referencia a producto.id_producto; los atributos propios de los insumos quedan en insumo. Un trigger impide que el mismo producto sea también una planta.

8. Normalización

Tabla	1FN	2FN	3FN	Observación
vendedor	Cumple	Cumple	Cumple	PK simple. El dni es clave candidata (UNIQUE). La dirección es un único texto.
cliente	Cumple	Cumple	Cumple	PK simple. El dni es clave candidata (UNIQUE) y admite letras (pasaportes).
proveedor	Cumple	Cumple	Cumple	PK simple. El cuit es clave candidata (UNIQUE).
producto	Cumple	Cumple	Cumple, con datos derivados	cantidad_stock y precio_compra (último precio pagado) se derivan de compras y ventas; los mantienen los triggers.
planta	Cumple	Cumple	Cumple	tipo_planta no determina de_interior, nivel_de_luz ni riego_cada_dias (hay suculentas de interior y de exterior).
insumo	Cumple	Cumple	Cumple	Sin dependencias entre atributos que no son clave.
venta	Cumple	Cumple	Cumple, con dato derivado	total_facturado es la suma de los subtotales de sus detalles.
detalle_venta	Cumple	Cumple	Cumple, con dato derivado	(id_venta, id_producto) es clave candidata (UNIQUE). subtotal = cantidad × precio_unitario. El precio_unitario es el de esa venta.
compra	Cumple	Cumple	Cumple, con dato derivado	total_facturado es la suma de los subtotales de sus detalles.
detalle_compra	Cumple	Cumple	Cumple, con dato derivado	(id_compra, id_producto) es clave candidata (UNIQUE). subtotal = cantidad × precio_unitario. El precio_unitario es el de esa compra.

8.1 Resumen del análisis de la normalización

1FN. Todas las tablas cumplen la primera forma normal: cada atributo guarda un único valor atómico, no hay grupos repetitivos (cada persona o empresa tiene un único teléfono) y toda tabla tiene clave primaria. La dirección del vendedor se guarda como un único texto, porque el sistema no la consulta por partes.

2FN. Todas las claves primarias son simples (un único id autoincremental), por lo que no pueden existir dependencias parciales. En detalle_venta y detalle_compra, la clave candidata compuesta (id_venta, id_producto) o (id_compra, id_producto) determina cantidad, precio_unitario y subtotal en conjunto: el precio_unitario es el de esa operación y no el vigente del producto, así que no depende solo del producto.

3FN. No hay dependencias entre atributos que no son clave, con una excepción deliberada: cuatro datos derivados que se guardan para no recalcularlos en cada consulta y para que un cambio posterior de precios no altere el historial. Son el subtotal de cada detalle (cantidad × precio_unitario), el total_facturado de ventas y compras (suma de subtotales), la cantidad_stock del producto (compras menos ventas) y su precio_compra (último precio pagado). Los triggers los recalculan en cada alta, modificación o baja de un renglón.

Conclusión. El modelo está normalizado hasta la tercera forma normal, con esos cuatro datos derivados como redundancia controlada. La separación de producto en producto, planta e insumo evita columnas nulas según el tipo de artículo y mantiene los datos botánicos solo donde corresponden.

9. Modelo Físico — MySQL

9.1 Tipos de datos y restricciones

Todos los atributos tienen un tipo de dato MySQL válido y un tamaño definido. Las restricciones se declaran en dos scripts: schema.sql (PK, NOT NULL, DEFAULT y UNIQUE) e integridad_referencial.sql (CHECK y claves foráneas, que se agregan con ALTER TABLE). Las tablas usan el motor InnoDB, el predeterminado de MySQL 8, que hace cumplir las claves foráneas y admite transacciones.

Restricción	Cantidad	Dónde se aplica
PRIMARY KEY	10	Una por tabla: id INT autoincremental.
FOREIGN KEY	9	Detalladas en la sección 5.2, con sus reglas ON DELETE / ON UPDATE.
UNIQUE	7	vendedor.dni, cliente.dni, proveedor.cuit, planta.id_producto, insumo.id_producto, (id_venta, id_producto) en detalle_venta y (id_compra, id_producto) en detalle_compra.
NOT NULL	48	Según la columna Nulabilidad de la sección 4.2.
CHECK	17	Formato de dni, teléfono, mail y cuit; stock, stock mínimo y precios mayores o iguales a 0; riego entre 1 y 365 días; totales mayores o iguales a 0; cantidad mayor a 0 y precio unitario mayor o igual a 0 en los detalles.
DEFAULT	8	cantidad_stock = 0, stock_minimo = 0, activo = TRUE, fecha = fecha del día (venta y compra), de_interior = false, total_facturado = 0 y subtotal = 0 (los calculan los triggers) y precio_compra = NULL.
ENUM	4	forma_de_pago (venta y compra), tipo_planta y nivel_de_luz.
Triggers	20	Subtipos de producto, fechas, subtotales, totales, stock y baja lógica (archivo triggers.sql).
Índices	7	Fechas de venta y compra, nombre de planta e insumo, tipo y nombre de planta, y apellido y nombre de cliente y vendedor (archivo indices.sql).

9.2 Script SQL de creación

```
DROP DATABASE IF EXISTS vivero;
CREATE DATABASE vivero CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE vivero;

-- ------------------------------------------------------
-- Personas y proveedores
-- ------------------------------------------------------

CREATE TABLE vendedor (
    id_vendedor INT AUTO_INCREMENT PRIMARY KEY,
    nombre      VARCHAR(50)  NOT NULL,
    apellido    VARCHAR(50)  NOT NULL,
    dni         VARCHAR(12)  NOT NULL UNIQUE,
```

```
    telefono    VARCHAR(15)  NOT NULL,
    mail        VARCHAR(100),
    direccion   VARCHAR(100)
);

-- Todos los datos son opcionales: el cliente puede no querer darlos.
-- El "consumidor final" es una fila más de esta tabla (ver datos.sql).
CREATE TABLE cliente (
    id_cliente  INT AUTO_INCREMENT PRIMARY KEY,
    nombre      VARCHAR(50),
    apellido    VARCHAR(50),
    dni         VARCHAR(12) UNIQUE,
    telefono    VARCHAR(15)
);

CREATE TABLE proveedor (
    id_proveedor   INT AUTO_INCREMENT PRIMARY KEY,
    nombre_empresa VARCHAR(100) NOT NULL,
    cuit           VARCHAR(11)  NOT NULL UNIQUE,      -- 11 dígitos, sin guiones
    telefono       VARCHAR(15)  NOT NULL
);

-- ------------------------------------------------------
-- Producto (supertipo): stock y precios de cada artículo
-- ------------------------------------------------------

CREATE TABLE producto (
    id_producto    INT AUTO_INCREMENT PRIMARY KEY,
    cantidad_stock INT           NOT NULL DEFAULT 0,  -- lo actualizan los
triggers
    stock_minimo   INT           NOT NULL DEFAULT 0,  -- por debajo de este valor
hay que reponer
    precio_compra  DECIMAL(10,2),                     -- precio de la última
compra; NULL = todavía sin compras
    precio_venta   DECIMAL(10,2) NOT NULL,
    activo         BOOLEAN       NOT NULL DEFAULT TRUE -- baja lógica: FALSE = no
se vende más
);

-- ------------------------------------------------------
-- Subtipos de producto
-- id_producto es UNIQUE: cada producto tiene a lo sumo una planta o un
-- insumo (relación 1:1). Que no esté en las dos tablas a la vez lo
-- controlan los triggers trg_planta_bi / trg_insumo_bi.
-- ------------------------------------------------------

CREATE TABLE planta (
    id_planta       INT AUTO_INCREMENT PRIMARY KEY,
    nombre          VARCHAR(50)  NOT NULL,
    tipo_planta     ENUM('suculenta', 'cactus', 'aromática', 'frutal',
                         'ornamental', 'arbusto', 'árbol', 'flor de estación') NOT
NULL,
    de_interior     BOOLEAN      NOT NULL DEFAULT FALSE,
    descripcion     VARCHAR(100),
    nivel_de_luz    ENUM('bajo', 'medio', 'alto') NOT NULL,
    riego_cada_dias SMALLINT     NOT NULL,
    id_producto     INT          NOT NULL UNIQUE
);

CREATE TABLE insumo (
```

```
    id_insumo   INT AUTO_INCREMENT PRIMARY KEY,
    nombre      VARCHAR(50)  NOT NULL,
    descripcion VARCHAR(100),
    id_producto INT          NOT NULL UNIQUE
);

-- ------------------------------------------------------
-- Circuito de venta
-- ------------------------------------------------------

CREATE TABLE venta (
    id_venta        INT AUTO_INCREMENT PRIMARY KEY,
    fecha           DATE NOT NULL DEFAULT (CURRENT_DATE), -- no puede ser futura:
trg_venta_bi/bu
    forma_de_pago   ENUM('efectivo', 'débito', 'crédito', 'transferencia') NOT
NULL,
    total_facturado DECIMAL(10,2) NOT NULL DEFAULT 0,     -- derivado: suma de
subtotales (trigger)
    id_cliente      INT NOT NULL,                         -- relación "recibe"
    id_vendedor     INT NOT NULL                          -- relación "realiza"
);

-- Relación N:M venta - producto
CREATE TABLE detalle_venta (
    id_detalle_venta INT AUTO_INCREMENT PRIMARY KEY,
    id_venta         INT NOT NULL,
    id_producto      INT NOT NULL,
    cantidad         INT NOT NULL,
    precio_unitario  DECIMAL(10,2) NOT NULL,              -- precio cobrado en
esta venta
    subtotal         DECIMAL(10,2) NOT NULL DEFAULT 0,    -- derivado: cantidad *
precio_unitario (trigger)
    -- un producto aparece una sola vez por venta (más unidades = más cantidad)
    UNIQUE KEY uq_dventa_venta_producto (id_venta, id_producto)
);

-- ------------------------------------------------------
-- Circuito de reposición
-- ------------------------------------------------------

CREATE TABLE compra (
    id_compra       INT AUTO_INCREMENT PRIMARY KEY,
    fecha           DATE NOT NULL DEFAULT (CURRENT_DATE), -- no puede ser futura:
trg_compra_bi/bu
    forma_de_pago   ENUM('efectivo', 'débito', 'crédito', 'transferencia') NOT
NULL,
    total_facturado DECIMAL(10,2) NOT NULL DEFAULT 0,     -- derivado: suma de
subtotales (trigger)
    id_proveedor    INT NOT NULL                          -- relación "provee"
);

-- Relación N:M compra - producto
CREATE TABLE detalle_compra (
    id_detalle_compra INT AUTO_INCREMENT PRIMARY KEY,
    id_compra         INT NOT NULL,
    id_producto       INT NOT NULL,
    cantidad          INT NOT NULL,
    precio_unitario   DECIMAL(10,2) NOT NULL,             -- precio pagado en esta
compra
```

```
    subtotal          DECIMAL(10,2) NOT NULL DEFAULT 0,   -- derivado: cantidad *
precio_unitario (trigger)
    UNIQUE KEY uq_dcompra_compra_producto (id_compra, id_producto)
);

-- ======================================================
-- 1. RESTRICCIONES DE DOMINIO
-- ======================================================

ALTER TABLE vendedor
  ADD CONSTRAINT chk_vendedor_dni     CHECK (dni REGEXP '^[0-9]+$'),
  ADD CONSTRAINT chk_vendedor_telefono CHECK (telefono REGEXP '^[+]?[0-9]+$'),
  ADD CONSTRAINT chk_vendedor_mail     CHECK (mail LIKE '%_@_%._%');

-- el dni admite letras: puede ser un pasaporte o un documento extranjero
ALTER TABLE cliente
  ADD CONSTRAINT chk_cliente_dni      CHECK (dni REGEXP '^[0-9A-Za-z]+$'),
  ADD CONSTRAINT chk_cliente_telefono CHECK (telefono REGEXP '^[+]?[0-9]+$');

ALTER TABLE proveedor
  ADD CONSTRAINT chk_proveedor_cuit     CHECK (cuit REGEXP '^[0-9]{11}$'),
  ADD CONSTRAINT chk_proveedor_telefono CHECK (telefono REGEXP '^[+]?[0-9]+$');

ALTER TABLE producto
  ADD CONSTRAINT chk_producto_stock   CHECK (cantidad_stock >= 0),
  ADD CONSTRAINT chk_producto_minimo  CHECK (stock_minimo >= 0),
  ADD CONSTRAINT chk_producto_precios CHECK (precio_compra >= 0 AND precio_venta >= 0);

ALTER TABLE planta
  ADD CONSTRAINT chk_planta_riego CHECK (riego_cada_dias BETWEEN 1 AND 365);

ALTER TABLE venta
  ADD CONSTRAINT chk_venta_total CHECK (total_facturado >= 0);

ALTER TABLE detalle_venta
  ADD CONSTRAINT chk_dventa_cantidad CHECK (cantidad > 0),
  ADD CONSTRAINT chk_dventa_precio   CHECK (precio_unitario >= 0);

ALTER TABLE compra
  ADD CONSTRAINT chk_compra_total CHECK (total_facturado >= 0);

ALTER TABLE detalle_compra
  ADD CONSTRAINT chk_dcompra_cantidad CHECK (cantidad > 0),
  ADD CONSTRAINT chk_dcompra_precio   CHECK (precio_unitario >= 0);

-- ======================================================
-- 2. CLAVES FORÁNEAS
-- ======================================================

-- ------------------------------------------------------
-- Subtipos de producto
-- ------------------------------------------------------
```

```
ALTER TABLE planta
   ADD CONSTRAINT fk_planta_producto
      FOREIGN KEY (id_producto) REFERENCES producto (id_producto)
      ON DELETE CASCADE ON UPDATE CASCADE;

ALTER TABLE insumo
   ADD CONSTRAINT fk_insumo_producto
      FOREIGN KEY (id_producto) REFERENCES producto (id_producto)
      ON DELETE CASCADE ON UPDATE CASCADE;

-- ------------------------------------------------------
-- Circuito de venta
-- ------------------------------------------------------

ALTER TABLE venta
   ADD CONSTRAINT fk_venta_cliente
      FOREIGN KEY (id_cliente) REFERENCES cliente (id_cliente)
      ON DELETE RESTRICT ON UPDATE CASCADE,
   ADD CONSTRAINT fk_venta_vendedor
      FOREIGN KEY (id_vendedor) REFERENCES vendedor (id_vendedor)
      ON DELETE RESTRICT ON UPDATE CASCADE;

ALTER TABLE detalle_venta
   ADD CONSTRAINT fk_dventa_venta
      FOREIGN KEY (id_venta) REFERENCES venta (id_venta)
      ON DELETE CASCADE ON UPDATE CASCADE,
   ADD CONSTRAINT fk_dventa_producto
      FOREIGN KEY (id_producto) REFERENCES producto (id_producto)
      ON DELETE RESTRICT ON UPDATE CASCADE;

-- ------------------------------------------------------
-- Circuito de reposición
-- ------------------------------------------------------

ALTER TABLE compra
   ADD CONSTRAINT fk_compra_proveedor
      FOREIGN KEY (id_proveedor) REFERENCES proveedor (id_proveedor)
      ON DELETE RESTRICT ON UPDATE CASCADE;

ALTER TABLE detalle_compra
   ADD CONSTRAINT fk_dcompra_compra
      FOREIGN KEY (id_compra) REFERENCES compra (id_compra)
      ON DELETE CASCADE ON UPDATE CASCADE,
   ADD CONSTRAINT fk_dcompra_producto
      FOREIGN KEY (id_producto) REFERENCES producto (id_producto)
      ON DELETE RESTRICT ON UPDATE CASCADE;


-- Reportes de ventas y compras por período (ej: ventas del mes)
CREATE INDEX idx_venta_fecha  ON venta (fecha);
CREATE INDEX idx_compra_fecha ON compra (fecha);
```

```
-- Búsqueda de productos por nombre (RNF-01)
CREATE INDEX idx_planta_nombre ON planta (nombre);
CREATE INDEX idx_insumo_nombre ON insumo (nombre);

-- Listados de plantas por categoría (RNF-01)
CREATE INDEX idx_planta_tipo_nombre ON planta (tipo_planta, nombre);

-- Búsqueda de personas por apellido y nombre
CREATE INDEX idx_cliente_apellido_nombre  ON cliente (apellido, nombre);
CREATE INDEX idx_vendedor_apellido_nombre ON vendedor (apellido, nombre);

DELIMITER $$

-- ======================================================
-- SUBTIPOS: un producto no puede ser planta e insumo a la vez
-- ======================================================

CREATE TRIGGER trg_planta_bi
BEFORE INSERT ON planta
FOR EACH ROW
BEGIN
   IF EXISTS (SELECT 1 FROM insumo WHERE id_producto = NEW.id_producto) THEN
      SIGNAL SQLSTATE '45000'
         SET MESSAGE_TEXT = 'El producto ya está registrado como insumo';
   END IF;
END$$

CREATE TRIGGER trg_planta_bu
BEFORE UPDATE ON planta
FOR EACH ROW
BEGIN
   IF NEW.id_producto <> OLD.id_producto
     AND EXISTS (SELECT 1 FROM insumo WHERE id_producto = NEW.id_producto) THEN
      SIGNAL SQLSTATE '45000'
         SET MESSAGE_TEXT = 'El producto ya está registrado como insumo';
   END IF;
END$$

CREATE TRIGGER trg_insumo_bi
BEFORE INSERT ON insumo
FOR EACH ROW
BEGIN
   IF EXISTS (SELECT 1 FROM planta WHERE id_producto = NEW.id_producto) THEN
      SIGNAL SQLSTATE '45000'
         SET MESSAGE_TEXT = 'El producto ya está registrado como planta';
   END IF;
END$$

CREATE TRIGGER trg_insumo_bu
BEFORE UPDATE ON insumo
FOR EACH ROW
BEGIN
```

```
    IF NEW.id_producto <> OLD.id_producto
      AND EXISTS (SELECT 1 FROM planta WHERE id_producto = NEW.id_producto) THEN
       SIGNAL SQLSTATE '45000'
          SET MESSAGE_TEXT = 'El producto ya está registrado como planta';
    END IF;
END$$

-- ======================================================
-- VENTA y COMPRA: fecha no futura y ajuste de stock al borrar
-- (un CHECK no puede usar CURRENT_DATE, por eso son triggers)
-- ======================================================

CREATE TRIGGER trg_venta_bi
BEFORE INSERT ON venta
FOR EACH ROW
BEGIN
    IF NEW.fecha > CURRENT_DATE THEN
       SIGNAL SQLSTATE '45000'
          SET MESSAGE_TEXT = 'La fecha de la venta no puede ser posterior a hoy';
    END IF;
END$$

CREATE TRIGGER trg_venta_bu
BEFORE UPDATE ON venta
FOR EACH ROW
BEGIN
    IF NEW.fecha > CURRENT_DATE THEN
       SIGNAL SQLSTATE '45000'
          SET MESSAGE_TEXT = 'La fecha de la venta no puede ser posterior a hoy';
    END IF;
END$$

CREATE TRIGGER trg_venta_bd
BEFORE DELETE ON venta
FOR EACH ROW
BEGIN
    -- devuelve al stock lo vendido; después la cascada borra los detalles
    UPDATE producto p
      JOIN detalle_venta dv ON dv.id_producto = p.id_producto
       SET p.cantidad_stock = p.cantidad_stock + dv.cantidad
     WHERE dv.id_venta = OLD.id_venta;
END$$

CREATE TRIGGER trg_compra_bi
BEFORE INSERT ON compra
FOR EACH ROW
BEGIN
    IF NEW.fecha > CURRENT_DATE THEN
       SIGNAL SQLSTATE '45000'
          SET MESSAGE_TEXT = 'La fecha de la compra no puede ser posterior a hoy';
    END IF;
END$$
```

```
CREATE TRIGGER trg_compra_bu
BEFORE UPDATE ON compra
FOR EACH ROW
BEGIN
   IF NEW.fecha > CURRENT_DATE THEN
      SIGNAL SQLSTATE '45000'
         SET MESSAGE_TEXT = 'La fecha de la compra no puede ser posterior a hoy';
   END IF;
END$$

CREATE TRIGGER trg_compra_bd
BEFORE DELETE ON compra
FOR EACH ROW
BEGIN
   -- descuenta lo que había sumado la compra; si ese stock ya se
   -- vendió, el CHECK chk_producto_stock frena el borrado
   UPDATE producto p
     JOIN detalle_compra dc ON dc.id_producto = p.id_producto
      SET p.cantidad_stock = p.cantidad_stock - dc.cantidad
    WHERE dc.id_compra = OLD.id_compra;
END$$

-- ======================================================
-- DETALLE_COMPRA
-- ======================================================

CREATE TRIGGER trg_detalle_compra_bi
BEFORE INSERT ON detalle_compra
FOR EACH ROW
BEGIN
   SET NEW.subtotal = NEW.cantidad * NEW.precio_unitario;
END$$

CREATE TRIGGER trg_detalle_compra_ai
AFTER INSERT ON detalle_compra
FOR EACH ROW
BEGIN
   -- suma stock y deja el precio de esta compra como último precio de compra
   UPDATE producto
      SET cantidad_stock = cantidad_stock + NEW.cantidad,
          precio_compra  = NEW.precio_unitario
    WHERE id_producto = NEW.id_producto;

   UPDATE compra
      SET total_facturado = (SELECT COALESCE(SUM(subtotal), 0)
                             FROM detalle_compra WHERE id_compra = NEW.id_compra)
    WHERE id_compra = NEW.id_compra;
END$$

CREATE TRIGGER trg_detalle_compra_bu
BEFORE UPDATE ON detalle_compra
```

```
FOR EACH ROW
BEGIN
   IF NEW.id_compra <> OLD.id_compra OR NEW.id_producto <> OLD.id_producto THEN
      SIGNAL SQLSTATE '45000'
         SET MESSAGE_TEXT = 'No se puede cambiar la compra ni el producto de un detalle; borrá el
detalle y creá uno nuevo';
   END IF;

   SET NEW.subtotal = NEW.cantidad * NEW.precio_unitario;
END$$

CREATE TRIGGER trg_detalle_compra_au
AFTER UPDATE ON detalle_compra
FOR EACH ROW
BEGIN
   -- ajusta el stock por la diferencia (si queda negativo, lo frena el CHECK de producto)
   -- y, si se corrigió el precio, lo toma como último precio de compra
   UPDATE producto
      SET cantidad_stock = cantidad_stock + (NEW.cantidad - OLD.cantidad),
         precio_compra  = NEW.precio_unitario
    WHERE id_producto = NEW.id_producto;

   UPDATE compra
      SET total_facturado = (SELECT COALESCE(SUM(subtotal), 0)
                        FROM detalle_compra WHERE id_compra = NEW.id_compra)
    WHERE id_compra = NEW.id_compra;
END$$

CREATE TRIGGER trg_detalle_compra_ad
AFTER DELETE ON detalle_compra
FOR EACH ROW
BEGIN
   -- revierte el stock que había sumado (si ya se vendió, lo frena el CHECK)
   UPDATE producto
      SET cantidad_stock = cantidad_stock - OLD.cantidad
    WHERE id_producto = OLD.id_producto;

   UPDATE compra
      SET total_facturado = (SELECT COALESCE(SUM(subtotal), 0)
                        FROM detalle_compra WHERE id_compra = OLD.id_compra)
    WHERE id_compra = OLD.id_compra;
END$$

-- ======================================================
-- DETALLE_VENTA
-- ======================================================

CREATE TRIGGER trg_detalle_venta_bi
BEFORE INSERT ON detalle_venta
FOR EACH ROW
BEGIN
   DECLARE v_stock  INT;
```

```
    DECLARE v_activo BOOLEAN;

    SELECT cantidad_stock, activo INTO v_stock, v_activo
      FROM producto WHERE id_producto = NEW.id_producto;

    IF NOT v_activo THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'El producto está dado de baja';
    END IF;

    IF v_stock < NEW.cantidad THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Stock insuficiente para el producto';
    END IF;

    SET NEW.subtotal = NEW.cantidad * NEW.precio_unitario;
END$$

CREATE TRIGGER trg_detalle_venta_ai
AFTER INSERT ON detalle_venta
FOR EACH ROW
BEGIN
    UPDATE producto
       SET cantidad_stock = cantidad_stock - NEW.cantidad
     WHERE id_producto = NEW.id_producto;

    UPDATE venta
       SET total_facturado = (SELECT COALESCE(SUM(subtotal), 0)
                              FROM detalle_venta WHERE id_venta = NEW.id_venta)
     WHERE id_venta = NEW.id_venta;
END$$

CREATE TRIGGER trg_detalle_venta_bu
BEFORE UPDATE ON detalle_venta
FOR EACH ROW
BEGIN
    DECLARE v_stock INT;

    IF NEW.id_venta <> OLD.id_venta OR NEW.id_producto <> OLD.id_producto THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'No se puede cambiar la venta ni el producto de un detalle; borrá el
detalle y creá uno nuevo';
    END IF;

    SELECT cantidad_stock INTO v_stock
      FROM producto WHERE id_producto = NEW.id_producto;

    -- si aumenta la cantidad, tiene que alcanzar el stock para la diferencia
    IF NEW.cantidad > OLD.cantidad AND v_stock < (NEW.cantidad - OLD.cantidad) THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Stock insuficiente para el producto';
    END IF;
```

```
    SET NEW.subtotal = NEW.cantidad * NEW.precio_unitario;
END$$

CREATE TRIGGER trg_detalle_venta_au
AFTER UPDATE ON detalle_venta
FOR EACH ROW
BEGIN
    UPDATE producto
       SET cantidad_stock = cantidad_stock - (NEW.cantidad - OLD.cantidad)
     WHERE id_producto = NEW.id_producto;

    UPDATE venta
       SET total_facturado = (SELECT COALESCE(SUM(subtotal), 0)
                              FROM detalle_venta WHERE id_venta = NEW.id_venta)
     WHERE id_venta = NEW.id_venta;
END$$

CREATE TRIGGER trg_detalle_venta_ad
AFTER DELETE ON detalle_venta
FOR EACH ROW
BEGIN
    -- devuelve el stock al borrar el detalle
    UPDATE producto
       SET cantidad_stock = cantidad_stock + OLD.cantidad
     WHERE id_producto = OLD.id_producto;

    UPDATE venta
       SET total_facturado = (SELECT COALESCE(SUM(subtotal), 0)
                              FROM detalle_venta WHERE id_venta = OLD.id_venta)
     WHERE id_venta = OLD.id_venta;
END$$

DELIMITER ;
```

10. Validación del Modelo

10.1 Casos de prueba

N°	Caso de prueba	Resultado esperado	Resultado obtenido
P1	Registrar una compra con una fila de detalle_compra (producto, cantidad y precio unitario).	El subtotal se calcula como cantidad × precio_unitario. El stock del producto aumenta en la cantidad comprada y su precio_compra pasa a ser el precio unitario. El total_facturado de la compra se recalcula.	En la tabla producto aumentó el stock de 9 a 19 y su precio_compra se mantuvo en $4500 ya que no varió el precio. En la tabla detalle_compra, se creó la fila del detalle con el subtotal de $45000. En la tabla compra, se creó la compra primeramente con el total_facturado en $0 que luego aumentó a $45000
P2	Registrar una venta con dos filas de detalle_venta.	El subtotal de cada fila se calcula como cantidad × precio_unitario. El stock de cada producto disminuye en la cantidad vendida y el total_facturado de la venta es la suma de los subtotales.	En la tabla venta, se creó la venta primeramente con el total_facturado en $0 que luego aumentó a $23000. En la tabla detalle_venta se crearon las dos filas con el subtotal de $10000 en la primera y $13000 en la segunda. En la tabla producto, el producto cuya id es 10, tuvo un descenso en la cantidad_stock de 164 a 154 y el producto cuya id es 16, tuvo un descenso en la cantidad_stock de 8 a 6.
P3	Borrar una venta completa.	Se eliminan sus detalles (ON DELETE CASCADE) y el stock vendido vuelve al producto (trg_venta_bd).	En la tabla venta desapareció la fila, en la tabla detalle_venta desaparecieron todas las filas de esa venta y en la tabla producto, cada producto aumentó su stock correspondiente. Es decir, el producto de id 10 aumentó su cantidad_stock nuevamente a 164 y el de id 16, aumentó su cantidad_stock a 8.
P4	Dar de baja un producto (activo = FALSE).	El producto permanece en la base con su historial y deja de poder venderse (ver 8).	Se mantiene el producto con su valor activo en 0 y las ventas y el detalle_venta ya realizadas sin modificar. Pero cuando se intenta realizar un nuevo detalle_venta con ese producto, salta el error (como N4): Error Code: 1644. El producto está dado de baja
N1	Vender más unidades que el stock disponible (100 unidades del producto 5).	Rechazado por el trigger trg_detalle_venta_bi: “Stock insuficiente para el producto”.	Error Code: 1644. Stock insuficiente para el producto
N2	Aumentar la cantidad de un detalle de venta por encima del stock (500 unidades).	Rechazado por el trigger trg_detalle_venta_bu: “Stock insuficiente para el producto”.	Error Code: 1644. Stock insuficiente para el producto
N3	Cambiar el producto de un detalle de venta ya cargado.	Rechazado por el trigger trg_detalle_venta_bu: “No se puede cambiar la venta ni el producto de un detalle…”.	Error Code: 1644. No se puede cambiar la venta ni el producto de un detalle; borrá el detalle y creá uno nuevo
N4	Vender un producto dado de baja.	Rechazado por el trigger trg_detalle_venta_bi: “El producto está dado de baja”.	Error Code: 1644. El producto está dado de baja
N5	Repetir un producto dentro de la misma venta.	Rechazado: “Duplicate entry” en la clave única uq_dventa_venta_producto.	Error Code: 1062. Duplicate entry '1-1' for key 'detalle_venta.uq_dventa_venta_producto'

N°	Caso de prueba	Resultado esperado	Resultado obtenido
N6	Anular un renglón de compra cuyo stock ya se vendió.	Rechazado por el CHECK chk_producto_stock: el stock quedaría negativo (trigger trg_detalle_compra_ad).	Error Code: 3819. Check constraint 'chk_producto_stock' is violated.
N7	Anular una compra completa cuyo stock ya se vendió.	Rechazado por el CHECK chk_producto_stock (trigger trg_compra_bd).	Error Code: 3819. Check constraint 'chk_producto_stock' is violated.
N8	Cargar como planta un producto que ya es insumo.	Rechazado por el trigger trg_planta_bi: “El producto ya está registrado como insumo”.	Error Code: 1644. El producto ya está registrado como insumo
N9	Cargar dos plantas para el mismo producto.	Rechazado: “Duplicate entry” en la clave única de planta.id_producto.	Error Code: 1062. Duplicate entry '1' for key 'planta.id_producto'
N10	Asignar a una planta una categoría fuera de las permitidas (“helecho”).	Rechazado: “Data truncated for column tipo_planta” (ENUM en modo estricto).	Error Code: 1265. Data truncated for column 'tipo_planta' at row 1
N11	Cargar un riego fuera de rango (0 días).	Rechazado por el CHECK chk_planta_riego.	Error Code: 3819. Check constraint 'chk_planta_riego' is violated.
N12	Cargar un precio de venta negativo.	Rechazado por el CHECK chk_producto_precios.	Error Code: 3819. Check constraint 'chk_producto_precios' is violated.
N13	Cargar un detalle de venta con cantidad 0.	Rechazado por el CHECK chk_dventa_cantidad.	Error Code: 3819. Check constraint 'chk_dventa_cantidad' is violated.
N14	Cargar una forma de pago fuera de las permitidas (“cheque”).	Rechazado: “Data truncated for column forma_de_pago” (ENUM en modo estricto).	Error Code: 1265. Data truncated for column 'forma_de_pago' at row 1
N15	Cargar una venta con fecha futura.	Rechazado por el trigger trg_venta_bi: “La fecha de la venta no puede ser posterior a hoy”.	Error Code: 1644. La fecha de la venta no puede ser posterior a hoy
N16	Cargar un cliente con un DNI ya registrado.	Rechazado: “Duplicate entry” en la clave única de dni.	Error Code: 1062. Duplicate entry '32456789' for key 'cliente.dni'
N17	Cargar un proveedor con un CUIT que no tiene 11 dígitos.	Rechazado por el CHECK chk_proveedor_cuit.	Error Code: 3819. Check constraint 'chk_proveedor_cuit' is violated.
N18	Cargar una venta de un vendedor que no existe.	Rechazado por la clave foránea fk_venta_vendedor.	Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails (`vivero`.`venta`, CONSTRAINT `fk_venta_vendedor` FOREIGN KEY (`id_vendedor`) REFERENCES `vendedor` (`id_vendedor`) ON UPDATE CASCADE)
N19	Borrar un cliente que tiene ventas.	Rechazado por la clave foránea fk_venta_cliente (ON DELETE RESTRICT).	Error Code: 1451. Cannot delete or update a parent row: a foreign key constraint fails (`vivero`.`venta`, CONSTRAINT `fk_venta_cliente` FOREIGN KEY (`id_cliente`) REFERENCES `cliente` (`id_cliente`) ON UPDATE CASCADE)
N20	Borrar un producto con ventas y compras.	Rechazado por una clave foránea con ON DELETE	Error Code: 1451. Cannot delete or update a parent row: a foreign key

N°	Caso de prueba	Resultado esperado	Resultado obtenido
		RESTRICT (fk_dcompra_producto o fk_dventa_producto). Debe darse de baja con activo = FALSE.	constraint fails (`vivero`.`detalle_compra`, CONSTRAINT `fk_dcompra_producto` FOREIGN KEY (`id_producto`) REFERENCES `producto` (`id_producto`) ON UPDATE CASCADE)

10.2 Consultas SQL de validación

N°	Qué responde	Técnicas SQL	Requerimiento
1	Top 3 vendedores por facturación.	JOIN, GROUP BY, COUNT, SUM, ORDER BY, LIMIT	RF-05
2	Top 3 productos más vendidos por unidades.	JOIN y LEFT JOIN a planta e insumo, COALESCE, GROUP BY, LIMIT	RF-05
3	Producto más facturado de cada tipo (planta e insumo).	UNION ALL de dos consultas con GROUP BY y LIMIT 1	RF-05
4	Ventas mensuales: cantidad, total y ticket promedio.	DATE_FORMAT, GROUP BY, COUNT, SUM, AVG, ROUND	RF-05
5	Facturación por forma de pago y su porcentaje sobre el total.	GROUP BY, subconsulta escalar, ROUND	RF-05
6	Top 3 clientes por monto comprado y total vendido a consumidor final.	JOIN, GROUP BY, UNION ALL, WHERE	RF-05
7	Alerta de stock crítico: productos activos por debajo de su stock mínimo.	LEFT JOIN, COALESCE, comparación entre columnas, ORDER BY	RF-04 y RF-05
8	Margen por producto vendido (precio cobrado contra precio de compra vigente).	JOIN, LEFT JOIN, SUM, cálculo de margen y porcentaje, GROUP BY	RF-05
9	Compras por proveedor.	LEFT JOIN, COUNT, SUM, MAX, GROUP BY	RF-05
10	Productos que nunca se vendieron.	Subconsulta correlacionada con NOT EXISTS, LEFT JOIN	RF-05
11	Detalle de una venta, como se imprimiría el ticket.	JOIN de seis tablas, CONCAT_WS, COALESCE	RF-03
12	Historial de compras de un cliente.	JOIN, LEFT JOIN, WHERE, ORDER BY	RF-05

11. Conclusión Técnica

Diseño final. El modelo final tiene diez tablas en MySQL 8.x: tres entidades de contacto (vendedor, cliente y proveedor), producto como supertipo con sus subtipos planta e insumo, y los circuitos de venta y de compra, cada uno con su cabecera y su detalle. Las relaciones N:M entre las transacciones y los productos se resolvieron con tablas intermedias (detalle_venta y detalle_compra), que además guardan el precio de cada operación. El modelo está normalizado hasta la tercera forma normal, con cuatro datos derivados conservados a propósito, y su integridad se garantiza con claves primarias y foráneas, restricciones CHECK y UNIQUE, valores por defecto y 20 triggers.

Requerimiento	Cómo se cumplió
RF-01	producto, planta e insumo permiten altas y modificaciones. Las bajas son lógicas (activo = FALSE) para conservar el historial de ventas y compras.
RF-02	Las tablas cliente y proveedor guardan sus datos con restricciones de formato para documento, CUIT y teléfono. El cliente “Consumidor final” permite registrar ventas a quien no se identifica.
RF-03	venta y detalle_venta registran cliente, vendedor, forma de pago, productos, cantidades y precios. Los subtotales y el total se calculan automáticamente.
RF-04	producto.cantidad_stock se actualiza con cada compra y venta mediante triggers y se consulta en tiempo real. No se puede vender sin stock ni un producto dado de baja.
RF-05	Las consultas 4 (ventas por mes), 12 (historial de compras por cliente) y 7 (stock crítico) cubren los reportes y las alertas pedidas.
RNF-01	Siete índices aceleran las búsquedas por nombre, categoría, apellido y fecha. El cálculo automático de totales y stock agiliza el procesamiento de la venta.
RNF-02	Motor InnoDB, claves primarias y foráneas con reglas ON DELETE / ON UPDATE, CHECK, UNIQUE y triggers. Las pruebas negativas N1 a N20 demuestran que se rechazan los datos inválidos.
RNF-03	Los tipos de dato elegidos (claves INT e importes DECIMAL(10,2)) admiten el volumen proyectado de 50.000 ventas anuales y 10.000 productos.

11.1 Mejoras futuras

- Hacer que la base garantice que todo producto tenga su fila en planta o en insumo, y que toda venta o compra tenga al menos un detalle. Hoy lo debe controlar la aplicación.
- Agregar un campo de tipo de documento en cliente (DNI, pasaporte u otro), porque la columna dni admite letras y hoy no distingue de qué documento se trata.
- Reemplazar los ENUM de forma de pago y categoría de planta por tablas de referencia si esas listas cambian con frecuencia.
- Validar el dígito verificador del CUIT, además de su formato.
- Definir usuarios y permisos por rol (por ejemplo vendedor y administrador) y registrar un historial de cambios.
- Incorporar descuentos, impuestos y pagos combinados en una misma venta, y probar el rendimiento con el volumen proyectado.